

Electronics Health Record

Multi-Host Hyperledger Fabric Network
Project Report

***Blockchain Technology / Practical Based Lab / Smart Contract
EHR***

National Institute of Technology Warangal

Department Of Computer Science Engineering

PRASOON KUMAR (25CSM2R15)

PARAG JAIN (25CSM2R27)

HARSH SONI (25CSM2S16)

Guided By

[Prof. Erukala Suresh Babu](#)

Table of Contents

1. Abstract

2. Introduction & Background

3. System Architecture

3.1 High-Level Architecture

3.2 Multi-Node Deployment Topology

3.3 Docker Swarm and Overlay Networking

3.4 Hyperledger Fabric Network Configuration

3.5 Transaction Lifecycle

3.6 Hybrid Storage Strategy (Blockchain + IPFS)

4. Individual Contributions

4.1 Machine 1 – Prasoon Kumar(25CSM2R15): Infrastructure Architect & Swarm Master

4.2 Machine 2 – Parag Jain (25CSM2R27): Smart Contract Lead & Clinical Portals

4.3 Machine 3 – Harsh Soni(25CSM2S16): API Gateway Architect

5. Code Listings

5.1 Chaincode (Smart Contract) – index.js

5.2 API Gateway – gateway.js

5.3 IPFS Integration – ipfs.js

5.4 Express REST Server – server.js

5.5 Docker Compose – Machine 1

5.6 Docker Compose – Machine 2

5.7 Docker Compose – Machine 3

6. Verification and Results

7. Conclusion and Future Scope

1. Abstract

This report presents the design, implementation, and evaluation of a Multi-Host Hyperledger Fabric network for managing Electronic Health Records (EHR) — specifically lab test results — across three physically separate machines. The system employs Hyperledger Fabric v2.x as the permissioned blockchain backbone, CouchDB as a rich-query-enabled state database, IPFS for off-chain heavy data storage, and a Node.js Express API gateway that acts as the single application entry-point. The three-peer topology ensures high availability, fault tolerance, and decentralised data ownership while complying with enterprise-grade TLS mutual authentication and MSP-based identity management.

Each physical machine was assigned a distinct engineering role: Machine 1 acts as the Orderer node and primary peer (peer0), Machine 2 hosts the second peer (peer1) and runs smart contract and clinical portal development, and Machine 3 hosts the third peer (peer2) together with the full API gateway and IPFS integration. Together the three machines form a single Fabric organisation (LabMSP) connected over a shared application channel (ehrchannel).

2. Introduction & Background

2.1 Problem Statement

Traditional laboratory information systems (LIS) store patient records in centralised relational databases. These architectures are prone to single-point-of-failure, data tampering, and lack of transparent audit trails — all critical concerns in healthcare. When multiple institutions (labs, hospitals, pharmacies) need to share records, inter-system trust becomes a major bottleneck.

2.2 Proposed Solution

We propose a permissioned blockchain network using Hyperledger Fabric. Unlike public blockchains, Fabric allows fine-grained access control, pluggable consensus, and off-chain storage. Each record write (lab result submission) is endorsed by peers, ordered by the orderer, and committed to all peers' ledgers, creating an immutable, auditable chain-of-custody for every lab result.

2.3 Objectives

- Deploy a 3-peer, 1-orderer Hyperledger Fabric network across 3 physical machines.
- Develop a smart contract (chaincode) for CRUD operations on lab result records.
- Implement a REST API gateway using Node.js and the Fabric Gateway SDK.
- Integrate IPFS for storing large payload data off-chain, keeping only CID references on-chain.
- Provide a web-based dashboard for querying and submitting lab results.
- Ensure TLS-encrypted inter-peer communication and MSP-based identity.

3. System Architecture

3.1 High-Level Architecture

The system follows a three-tier architecture:

Tier 1 – Blockchain Layer: Three Hyperledger Fabric peers (peer0, peer1, peer2) and one Raft orderer, each running in Docker containers. All peers belong to LabMSP and are joined to the channel 'ehrchannel'.

Tier 2 – Storage Layer: Each peer is backed by a CouchDB instance (on the same host) that stores the world-state. Large medical payloads are stored on IPFS; only their CID (Content Identifier) and a SHA-256 digest are stored on-chain.

Tier 3 – Application Layer: Node.js Express server exposes a REST API. The Fabric Gateway SDK connects the application to any chosen peer via gRPC+TLS. A web dashboard (served as static files) consumes this API.

3.2 Multi-Node Deployment Topology

Machine	Hostname / IP	Containers	Role
Machine 1	10.166.46.138 (macOS + Colima)	orderer.example.com peer0.lab.example.com couchdb-peer0	Orderer + Primary Peer Channel Creator
Machine 2	10.166.46.204 (Ubuntu)	peer1.lab.example.com couchdb-peer1	Secondary Peer Smart Contract Dev
Machine 3	10.166.46.38 (Ubuntu)	peer2.lab.example.com couchdb-peer2	Tertiary Peer API Gateway Host

Shared	All machines	ehrchannel	Application channel with LabMSP
--------	--------------	------------	---------------------------------

3.3 Docker Swarm and Overlay Networking

Each machine runs its peer within a Docker bridge network named 'fabric-net'. The containers communicate across machines using static /etc/hosts entries (or extra_hosts in docker-compose). The peers resolve each other's hostnames using these mappings, enabling gRPC-based gossip propagation and block synchronisation. TLS certificates are pre-generated via cryptogen and distributed to all machines before startup.

3.4 Hyperledger Fabric Network Configuration

3.4.1 Organisation and Channel

One application organisation: LabMSP. One application channel: ehrchannel. The genesis block is generated from configtx.yaml and stored as system-genesis-block/genesis.block.

3.4.2 Consensus Mechanism

The Raft consensus algorithm is used via the Fabric built-in orderer. Raft is crash-fault-tolerant and suitable for enterprise deployments with a known, permissioned set of participants.

3.4.3 Certificate Authority (CA)

cryptogen is used to bootstrap cryptographic material (MSP certs, TLS certs) for all peers, the orderer, and users. All certificates are generated on Machine 1 and rsync'd to Machines 2 and 3.

3.4.4 Endorsement Policy

Default endorsement: any peer in LabMSP can endorse. This means a single peer endorsement is sufficient, while all peers maintain the same ledger state through ordering and committing.

3.5 Transaction Lifecycle

Read (Query) transactions:

- Client sends `evaluateTransaction()` to `gateway.js` via the REST API.
- Gateway connects to the target peer (`peer0/1/2`) via gRPC.
- Peer executes chaincode in simulation, returns result directly without ordering.
- Result is returned to client as JSON.

Write (Invoke) transactions:

- Client sends `submitTransaction()` to the gateway.
- Gateway submits proposal to endorsing peer; peer simulates chaincode, signs response.
- Endorsed proposal is sent to the orderer (`orderer.example.com:7050`).
- Orderer batches and orders transactions into a block, broadcasts to all peers.
- Peers validate and commit the block; CouchDB world-state is updated.
- Commit status is returned to the client.

3.6 Hybrid Storage Strategy (Blockchain + IPFS)

Large lab result payloads (images, extended test panels) are stored on IPFS via `ipfs.js`. The IPFS API is called at `http://127.0.0.1:5001`. Only the CID, SHA-256 digest, byte-length, and gateway URL are stored on-chain inside the `resultData` field. This pattern avoids blockchain bloat while maintaining data integrity through cryptographic hashing.

4. Individual Contributions

4.1 Machine 1 – Prasoon Kumar (25CSM2R27)

Role: Infrastructure Architect & Swarm Master

4.1.1 Role Overview

Prasoon was responsible for setting up the entire Hyperledger Fabric network bootstrap infrastructure on Machine 1 (macOS + Colima). His work formed the backbone that Machines 2 and 3 depend on. Without the orderer, genesis block, and channel artifacts that Prasoon configured, neither peer1 nor peer2 could join the network.

4.1.2 Containers Hosted on Machine 1

- orderer.example.com – Raft orderer, listens on port 7050
- peer0.lab.example.com – Primary peer, listens on port 7051 (chaincode port 7052)
- couchdb-peer0.lab.example.com – CouchDB world-state database for peer0, port 5984

4.1.3 Network Initialisation & Channel Creation

Prasoon ran the following sequence on Machine 1:

```
./scripts/use-colima.sh          # configure  
Docker runtime to Colima  
  
./scripts/download-fabric.sh    # download Fabric  
v2.x binaries & Docker images  
  
./scripts/generate-artifacts.sh  # cryptogen +  
configtxgen → genesis block + channel tx  
  
./scripts/start-machine1.sh     # docker-compose  
up orderer + peer0 + couchdb-peer0  
  
./scripts/create-channel.sh     # peer channel  
create + peer0 joins ehrchannel
```

4.1.4 Chaincode Deployment (deploy-labresults-machine1.sh)

Prasoon orchestrated the full chaincode lifecycle on behalf of the organisation:

- Package: peer lifecycle chaincode package labresults.tar.gz
- Install on peer0: peer lifecycle chaincode install labresults.tar.gz
- Approve for LabMSP: peer lifecycle chaincode approveformyorg
- Commit to channel: peer lifecycle chaincode commit
- Invoke InitLedger: pre-populate the ledger with two seed lab results
- Query all results to confirm successful deployment

4.1.5 Cryptographic Material Distribution

After generating all crypto material via cryptogen (organisations/ directory), Prasoon rsync'd the full repo to Machines 2 and 3. This included TLS certs, MSP configs, the ehrchannel.block genesis block, and the packaged chaincode – all pre-built so Machines 2 and 3 only need to start their peer and join the channel.

```
rsync -av --delete /laborg/  
user@MACHINE2:~/laborg/  
rsync -av --delete /laborg/  
user@MACHINE3:~/laborg/
```

4.1.6 Docker Compose Configuration (Machine 1)

File: docker/docker-compose.machine1.yaml

Key settings:

- ORDERER_GENERAL_GENESISMETHOD=file → reads genesis.block from volume mount
- ORDERER_GENERAL_TLS_ENABLED=true → mutual TLS for all orderer communications
- CORE_LEDGER_STATE_STATEDATABASE=CouchDB → world-state in CouchDB

- CORE_PEER_GOSSIP_EXTERNALENDPOINT=peer0.lab.example.com:7051 → gossip anchor
- CORE_VM_ENDPOINT=unix:///host/var/run/docker.sock → chaincode container launch
- Ports exposed: 7050 (orderer), 7051 (peer), 7052 (chaincode), 9444 (metrics/ops)

4.1.7 Verification Commands

Verify all three peers joined the channel from Machine 1:

```
./scripts/verify-network.sh
./scripts/verify-channel.sh peer0
./scripts/query-labresults.sh peer0
GetAllLabResults
```

Results of Machine 1 Verification:

Enter Machine 1 Verification Results Here:

```
praseonk@Praseons-MacBook-Air laborg % ./scripts/verify-network.sh
on: context deadline exceeded
praseonk@Praseons-MacBook-Air laborg % ./scripts/verify-network.sh

=== peer0 ===
Checking peer0 on channel ehrchannel ...
2026-04-24 21:43:06.210 IST 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
Blockchain info: {"height":45,"currentBlockHash":"vBj9XLh8A1LZ1IhzMPT300gUpJmLeilQfsY2My1SU=", "previousBlockHash":"ff17uTk0xAtC3pTpcLMd1g1bME0foJ0wzIQf027JT1U="}

=== peer1 ===
Checking peer1 on channel ehrchannel ...
2026-04-24 21:43:06.434 IST 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
Blockchain info: {"height":45,"currentBlockHash":"vBj9XLh8A1LZ1IhzMPT300gUpJmLeilQfsY2My1SU=", "previousBlockHash":"ff17uTk0xAtC3pTpcLMd1g1bME0foJ0wzIQf027JT1U="}

=== peer2 ===
Checking peer2 on channel ehrchannel ...
Error: error getting endorser client for channel: endorser client failed to connect to peer2.lab.example.com:7051: failed to create new connection: context deadline exceeded
praseonk@Praseons-MacBook-Air laborg % ./scripts/verify-network.sh

=== peer0 ===
Checking peer0 on channel ehrchannel ...
2026-04-24 21:45:04.754 IST 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
Blockchain info: {"height":45,"currentBlockHash":"vBj9XLh8A1LZ1IhzMPT300gUpJmLeilQfsY2My1SU=", "previousBlockHash":"ff17uTk0xAtC3pTpcLMd1g1bME0foJ0wzIQf027JT1U="}

=== peer1 ===
Checking peer1 on channel ehrchannel ...
2026-04-24 21:45:04.852 IST 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
Blockchain info: {"height":45,"currentBlockHash":"vBj9XLh8A1LZ1IhzMPT300gUpJmLeilQfsY2My1SU=", "previousBlockHash":"ff17uTk0xAtC3pTpcLMd1g1bME0foJ0wzIQf027JT1U="}

=== peer2 ===
Checking peer2 on channel ehrchannel ...
2026-04-24 21:45:36.577 IST 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
Blockchain info: {"height":47,"currentBlockHash":"51SzkzRD0J+k3jYfInBg9Yod8Uq2vzyknlPVV1MTVoE=", "previousBlockHash":"JA1W50vwZ9bT1rANp1svVnLOG2svCtG1/W2wP7dwhc="}

=== peer0 ===
Checking peer0 on channel ehrchannel ...
2026-04-24 21:58:36.658 IST 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
Blockchain info: {"height":47,"currentBlockHash":"51SzkzRD0J+k3jYfInBg9Yod8Uq2vzyknlPVV1MTVoE=", "previousBlockHash":"JA1W50vwZ9bT1rANp1svVnLOG2svCtG1/W2wP7dwhc="}

=== peer1 ===
Checking peer1 on channel ehrchannel ...
2026-04-24 21:58:36.741 IST 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
Blockchain info: {"height":47,"currentBlockHash":"51SzkzRD0J+k3jYfInBg9Yod8Uq2vzyknlPVV1MTVoE=", "previousBlockHash":"JA1W50vwZ9bT1rANp1svVnLOG2svCtG1/W2wP7dwhc="}

=== peer2 ===
Checking peer2 on channel ehrchannel ...
2026-04-24 21:58:36.741 IST 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
Blockchain info: {"height":47,"currentBlockHash":"51SzkzRD0J+k3jYfInBg9Yod8Uq2vzyknlPVV1MTVoE=", "previousBlockHash":"JA1W50vwZ9bT1rANp1svVnLOG2svCtG1/W2wP7dwhc="}
praseonk@Praseons-MacBook-Air laborg %
```

4.2 Machine 2 – Parag Jain (25CSM2R27)

Role: Smart Contract Lead & Clinical Portals

4.2.1 Role Overview

Parag was responsible for the design and implementation of the Fabric smart contract (chaincode) deployed on ehrchannel, and for the web-based portals (Patient Portal and Pharmacist/Billing Portal). He also authored the CouchDB rich-query selectors and the identity resolution algorithm that maps incoming user requests to their appropriate LabMSP roles.

4.2.2 Containers Hosted on Machine 2

- peer1.lab.example.com – Secondary peer, listens on port 7051
- couchdb-peer1.lab.example.com – CouchDB state database for peer1, port 5984

4.2.3 Machine 2 Setup Sequence

```
./scripts/use-docker-linux.sh    # verify Docker
Engine on Ubuntu

./scripts/download-fabric.sh    # download
Linux Fabric binaries

./scripts/start-machine2.sh      #
docker-compose up peer1 + couchdb-peer1

./scripts/join-channel.sh peer1 # peer1 joins
ehrchannel using ehrchannel.block

./scripts/install-chaincode.sh peer1 # install
pre-packaged labresults chaincode
```

4.2.4 Smart Contract Architecture (Chaincode) – index.js

Mohit authored the LabResultsContract class extending Hyperledger Fabric's Contract base class. The chaincode exposes the following transaction functions:

Function	Type	Description
InitLedger(ctx)	Invoke	Seeds two default lab results if ledger is empty
CreateLabResult(ctx, ...)	Invoke	Creates a new lab result record on-chain; errors if duplicate
ReadLabResult(ctx, resultId)	Query	Retrieves a single lab result by ID
UpdateLabResult(ctx, ...)	Invoke	Replaces all fields of an existing lab result
UpdateLabStatus(ctx, resultId, status)	Invoke	Updates only the status field + records TX ID
DeleteLabResult(ctx, resultId)	Invoke	Removes record from state; returns deleted=true
GetLabResultsByPatient(ctx, patientId)	Query	CouchDB rich query: selector by docType + patientId
GetAllLabResults(ctx)	Query	Range scan + filter by docType='labResult'

4.2.5 Identity Resolution Algorithm

Each record stored on-chain includes an ownerOrg field set to 'LabMSP'. The identity resolution algorithm ensures that only members of LabMSP (identified by their MSP certificate) can submit transactions.

The `ctx.clientIdentity.getMSPID()` function is used in production to enforce organisation-level access.

4.2.6 CouchDB Rich Queries

Mohit implemented Mango query selectors for CouchDB-backed peers:

```
{
  "selector": {
    "docType": "labResult",
    "patientId": "<patientId>"
  }
}
```

This enables efficient patient-scoped queries without scanning the entire ledger, critical for hospital workflows with millions of records.

4.2.7 Verification Commands (Machine 2)

```
./scripts/verify-channel.sh peer1
./scripts/query-labresults.sh peer1
GetAllLabResults
./scripts/query-labresults.sh peer1
GetLabResultsByPatient patient-1001
```

Results of Machine 2 Verification:

Enter Machine 2 Verification Results Here:

```
client > node-gateway > gateway.js > ...
1 'use strict';
2
3 const crypto = require('crypto');
4 const fs = require('fs');
5 const path = require('path');
6 const grpc = require('grpc/grpc-js');
7 const { connect, signers } = require('@hyperledger/fabric-gateway');
8
9 const ROOT_DIR = path.resolve(__dirname, '..', '..');
10 const CEP_PATH = path.join(ROOT_DIR, 'connection-profiles', 'lab-connection.json');
11 const USER_MSP_DIR = path.join(
12   ROOT_DIR,
13   'connection-profiles',
14   'user',
15   'msp');
16
17 parag-jain@parag-jain-HP-Pavilion-Laptop-14-dv6xxx:~/Downloads/laborg/laborg$ ./scripts/start-machine2.sh
Container peer1.lab.example.com Started
NAME STATUS PORTS
peer1.lab.example.com Up Less than a second 0.0.0.0:7051->7051/tcp, [::]:7051->7051/tcp
couchdb-peer1.lab.example.com Up Less than a second 4369/tcp, 5984/tcp, 9100/tcp
parag-jain@parag-jain-HP-Pavilion-Laptop-14-dv6xxx:~/Downloads/laborg/laborg$ ./scripts/verify-network.sh
=== peer0 ===
Checking peer0 on channel ehrchannel ...
2026-04-24 21:43:02.657 IST [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
Blockchain info: {"height":45,"currentBlockHash":"vVBj9XLH8A11ZiHzMpT300upJmLc1lqfsY2WY15U=", "previousBlockHash":"FF17uTK8AtC3pTcLM1g1ME0foJ0wz1QF0277T1U="}
=== peer1 ===
Checking peer1 on channel ehrchannel ...
2026-04-24 21:43:02.745 IST [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
Blockchain info: {"height":45,"currentBlockHash":"vVBj9XLH8A11ZiHzMpT300upJmLc1lqfsY2WY15U=", "previousBlockHash":"FF17uTK8AtC3pTcLM1g1ME0foJ0wz1QF0277T1U="}
=== peer2 ===
Checking peer2 on channel ehrchannel ...
Error: error getting endorser client for channel: endorser client failed to connect to peer2.lab.example.com:7051: failed to create new connection: context deadline exceeded
parag-jain@parag-jain-HP-Pavilion-Laptop-14-dv6xxx:~/Downloads/laborg/laborg$ ./scripts/verify-network.sh
=== peer0 ===
Checking peer0 on channel ehrchannel ...
2026-04-24 22:13:24.482 IST [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
Blockchain info: {"height":47,"currentBlockHash":"S15zkRD03k3jYfInBg9YodUqZvysknlPVV3MTVoB=", "previousBlockHash":"3A1u050wZ9BT1rNp1svVnL002svctgl/W2wP7dwhc="}
=== peer1 ===
Checking peer1 on channel ehrchannel ...
2026-04-24 22:13:24.619 IST [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
Blockchain info: {"height":47,"currentBlockHash":"S15zkRD03k3jYfInBg9YodUqZvysknlPVV3MTVoB=", "previousBlockHash":"3A1u050wZ9BT1rNp1svVnL002svctgl/W2wP7dwhc="}
=== peer2 ===
Checking peer2 on channel ehrchannel ...
Error: error getting endorser client for channel: endorser client failed to connect to peer2.lab.example.com:7051: failed to create new connection: connection error: des
c = "transport: error while dialing: dial tcp 19.168.46.38:7051: connect: connection refused"
parag-jain@parag-jain-HP-Pavilion-Laptop-14-dv6xxx:~/Downloads/laborg/laborg$
```

4.3 Machine 3 – Harsh Soni(25CSM2S16)

Role: API Gateway Architect

4.3.1 Role Overview

Harsh configured Machine 3 as the third peer (peer2) and built the complete Node.js API gateway layer, integrating the Hyperledger Fabric network with an IPFS-backed hybrid storage strategy and a REST API consumed by the web dashboard. He also resolved the critical network connectivity challenges between the Ubuntu host and Fabric containers, patching the gRPC connection logic to use static IP mappings when DNS resolution fails.

4.3.2 Containers Hosted on Machine 3

- peer2.lab.example.com – Tertiary peer, listens on port 7051
- couchdb-peer2.lab.example.com – CouchDB state database for peer2, port 5984

4.3.3 Machine 3 Setup Sequence

```
./scripts/use-docker-linux.sh    # verify Docker
Engine on Ubuntu

./scripts/download-fabric.sh    # download
Linux Fabric binaries

./scripts/start-machine3.sh      #
docker-compose up peer2 + couchdb-peer2

./scripts/join-channel.sh peer2 # peer2 joins
ehchannel

./scripts/install-chaincode.sh peer2 # install
labresults chaincode on peer2
```

4.3.4 API Gateway Architecture (gateway.js)

Ronit built gateway.js using the @hyperledger/fabric-gateway SDK. Key design decisions:

- Static IP mapping (STATIC_IPS object) overrides DNS resolution for peer hostnames, solving the container DNS issue on Machine 3 where peer hostnames were not resolvable from the host.
- grpc.ssl_target_name_override ensures TLS certificate hostname validation still works even when connecting via raw IP address.
- Deadline timeouts: evaluate=5s, endorse=15s, submit=15s, commitStatus=60s — tuned for LAN latency.
- withContract() abstraction opens/closes the gRPC connection and Fabric Gateway per request, preventing connection leaks.
- GATEWAY_PEER env variable allows dynamic peer selection without code changes.

4.3.5 Smart Routing Algorithm

The gateway supports per-request peer selection via the ?peer= query parameter or the X-Gateway-Peer HTTP header. This allows read-heavy

queries to be distributed across all three peers for load balancing, while writes are routed to the endorsing peer.

```
# Query peer2 directly:
GET /api/records?peer=peer2.lab.example.com

# Use X-Gateway-Peer header:
GET /api/records X-Gateway-Peer:
peer1.lab.example.com
```

4.3.6 Wallet and Identity Management

Harsh integrated pre-enrolled User1@lab.example.com identity from the MSP directory. The private key is loaded from organizations/peerOrganizations/lab.example.com/users/User1@lab.example.com/msp/keystore/ and the signing certificate from the signcerts directory. No enrollment CA step is needed, simplifying deployment for the academic prototype.

4.3.7 High Availability and Peer Redundancy

docker-compose.machine3.yaml includes extra_hosts entries that map all peer and orderer hostnames to their static IPs, ensuring that even if /etc/hosts is not configured on the Docker host, the containers themselves can resolve peer addresses.

```
extra_hosts:
  - "orderer.example.com:10.166.46.138"
  - "peer0.lab.example.com:10.166.46.138"
  - "peer1.lab.example.com:10.166.46.204"
```

4.3.8 IPFS Integration (ipfs.js)

Ronit implemented the IPFS module that offloads large lab result payloads:

- addJsonToIpfs(): serialises payload to JSON, posts to IPFS API /api/v0/add, returns CID + SHA-256 digest

- readJsonFromIpfs(): fetches content by CID from /api/v0/cat
- buildGatewayUrl(): constructs public gateway URL (http://127.0.0.1:8080/ipfs/<CID>)
- ipfsHealth(): probes the local IPFS daemon and returns version info
- pin=true ensures data persists across IPFS GC cycles

4.3.9 REST API Endpoints (server.js)

Method + Path	Description	Storage
GET /api/meta	List peers, channel, chaincode, IPFS config	—
GET /api/health	Application health check	—
GET /api/ipfs/health	IPFS daemon health check	IPFS
POST /api/ipfs/add-json	Upload JSON payload to IPFS	IPFS
GET /api/ipfs/json/:cid	Retrieve JSON by CID from IPFS	IPFS
GET /api/records	Get all lab results from ledger	Blockchain
GET /api/records/:resultId	Get a single lab result by ID	Blockchain
GET /api/patients/:patientId/records	Get all results for a patient	Blockchain
POST /api/records	Create new lab result (on-chain)	Blockchain
POST /api/records/ipfs	Create result with payload on IPFS, CID on-chain	Both
PATCH /api/records/:resultId/status	Update result status	Blockchain

GET /api/records/:id/res olve-data	Read record + fetch IPFS payload if stored off-chain	Both
--	--	------

4.3.10 Verification Commands (Machine 3)

```
./scripts/verify-channel.sh peer2
./scripts/query-labresults.sh peer2
GetAllLabResults

# Test API gateway:
curl http://localhost:3000/api/records
curl http://localhost:3000/api/meta
curl http://localhost:3000/api/health
```

Results of Machine 3 Verification:

Enter Machine 3 Verification Results Here:

```

harsh@harsh-GF63-Thin-95CXR:~/Downloads/Laborg$ cd laborg
harsh@harsh-GF63-Thin-95CXR:~/Downloads/Laborg$ ./scripts/start-machine3.sh
[*] up 3/3
  ✓ Network fabric labnet
  ✓ Container couchdb-peer2.lab.example.com Created 0.1s
  ✓ Container peer2.lab.example.com Created 0.1s
NAMEs
peer2.lab.example.com Up Less than a second 0.0.0.0:7051->7051/tcp, [::]:7051->7051/tcp
couchdb-peer2.lab.example.com Up 1 second 4369/tcp, 5984/tcp, 9100/tcp
harsh@harsh-GF63-Thin-95CXR:~/Downloads/Laborg$ ./scripts/verify-network.sh

=== peer0 ===
Checking peer0 on channel ehrchannel ...
2020-04-24 21:44:54.130 IST INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
Blockchain info: {"height":45,"currentBlockHash":"vVBj9XLh8A1LZIhZMpT300gUpJmLe1lqfsY2My15U=", "previousBlockHash":"fFL7uTk8xATC3pTpcLMD1g1bME0foJ0wzIQF027JT1U="}

=== peer1 ===
Checking peer1 on channel ehrchannel ...
2020-04-24 21:44:54.283 IST INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
Blockchain info: {"height":45,"currentBlockHash":"vVBj9XLh8A1LZIhZMpT300gUpJmLe1lqfsY2My15U=", "previousBlockHash":"fFL7uTk8xATC3pTpcLMD1g1bME0foJ0wzIQF027JT1U="}

=== peer2 ===
Checking peer2 on channel ehrchannel ...
2020-04-24 21:44:54.389 IST INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
Blockchain info: {"height":45,"currentBlockHash":"vVBj9XLh8A1LZIhZMpT300gUpJmLe1lqfsY2My15U=", "previousBlockHash":"fFL7uTk8xATC3pTpcLMD1g1bME0foJ0wzIQF027JT1U="}
harsh@harsh-GF63-Thin-95CXR:~/Downloads/Laborg$ docker ps
CONTAINER ID        IMAGE               PORTS
STATUS
d939e9c244eb      dev-peer2.lab.example.com-labresults_2-d9b294cbed11f932e053468bd2c664a171c84b51115078a957c93b702685362-707ae9e6f0b3e02efeb8a9c81e591e82a269d07f055eb6e6e8c9abcb5d5a6992f  18 seconds
ago Up 18 seconds  dev-peer2.lab.example.com-labresults_2-d9b294cbed11f932e053468bd2c664a171c84b51115078a957c93b702685362  "docker-entrypoint.s..."
6665c6b8a426      hyperledger/fabric-peer:2.5.15  0.0.0.0:7051->7051/tcp, [::]:7051->7051/tcp  peer2.lab.example.com  27 seconds
ago Up 26 seconds  couchdb:3.3  couchdb-peer2.lab.example.com  "tini -- /docker-ent..."
7c8214965f6a      couchdb:3.3
ago Up 26 seconds  4369/tcp, 5984/tcp, 9100/tcp
harsh@harsh-GF63-Thin-95CXR:~/Downloads/Laborg$

```

5. Code Listings

5.1 Chaincode (Smart Contract) – [chaincode/lab-results/index.js](#)

Author: Parag Jain (Machine 2) | Language: Node.js
(fabric-contract-api)

```
'use strict';

const { Contract } =
require('fabric-contract-api');

class LabResultsContract extends Contract {
  async ResultExists(ctx, resultId) {
    const data = await
ctx.stub.getState(resultId);
    return data && data.length > 0;
  }

  async InitLedger(ctx) {
    const existing = await
this.ResultExists(ctx, 'labresult1');
    if (existing) {
      return JSON.stringify({ message: 'Ledger
already initialized', initialized: false });
    }
    const seedRecords = [
      { resultId: 'labresult1', patientId:
'patient-1001', testCode: 'CBC',
        collectedAt: '2026-04-19T08:00:00Z',
status: 'REPORTED',
        resultData: { hemoglobin: '13.8', wbc:
'6200', platelets: '250000' } },
      { resultId: 'labresult2', patientId:
'patient-1002', testCode: 'LFT',
```

```

        collectedAt: '2026-04-19T09:30:00Z',
status: 'IN_REVIEW',
        resultData: { alt: '42', ast: '37',
bilirubin: '0.8' } },
    ];
    for (const record of seedRecords) {
        const payload =
this._buildRecord(record);
        await ctx.stub.putState(record.resultId,
Buffer.from(JSON.stringify(payload)));
    }
    return JSON.stringify({ message: 'Ledger
initialized', initialized: true, count: 2 });
}

async CreateLabResult(ctx, resultId,
patientId, testCode, collectedAt, status,
resultDataJson) {
    const exists = await this.ResultExists(ctx,
resultId);
    if (exists) throw new Error(`Lab result
${resultId} already exists`);
    const record = this._buildRecord({
resultId, patientId, testCode, collectedAt,
status,
        resultData:
this._parseJson(resultDataJson) });
    await ctx.stub.putState(resultId,
Buffer.from(JSON.stringify(record)));
    return JSON.stringify(record);
}

async ReadLabResult(ctx, resultId) {

```

```

        const data = await
ctx.stub.getState(resultId);
        if (!data || data.length === 0) throw new
Error(`Lab result ${resultId} does not exist`);
        return data.toString();
    }

    async UpdateLabStatus(ctx, resultId, status)
{
        const current = JSON.parse(await
this.ReadLabResult(ctx, resultId));
        current.status = status;
        current.updatedByTx = ctx.stub.getTxID();
        await ctx.stub.putState(resultId,
Buffer.from(JSON.stringify(current)));
        return JSON.stringify(current);
    }

    async DeleteLabResult(ctx, resultId) {
        const exists = await this.ResultExists(ctx,
resultId);
        if (!exists) throw new Error(`Lab result
${resultId} does not exist`);
        await ctx.stub.deleteState(resultId);
        return JSON.stringify({ deleted: true,
resultId });
    }

    async GetLabResultsByPatient(ctx, patientId)
{
        const query = { selector: { docType:
'labResult', patientId } };
        const iterator = await

```

```

ctx.stub.getQueryResult(JSON.stringify(query));
    const results = await
this._collectIterator(iterator);
    return JSON.stringify(results);
}

async GetAllLabResults(ctx) {
    const iterator = await
ctx.stub.getStateByRange('', '');
    const results = await
this._collectIterator(iterator);
    return JSON.stringify(results.filter(r =>
r.docType === 'labResult'));
}

_buildRecord({ resultId, patientId, testCode,
collectedAt, status, resultData }) {
    return { docType: 'labResult', resultId,
patientId, testCode, collectedAt,
        status, resultData, ownerOrg: 'LabMSP',
updatedByTx: null };
}

_parseJson(jsonText) {
    try { return JSON.parse(jsonText); }
    catch (e) { throw new Error(`Invalid JSON
payload: ${e.message}`); }
}

async _collectIterator(iterator) {
    const results = [];
    while (true) {
        const item = await iterator.next();

```

```
        if (item.value && item.value.value) {
            try {
results.push(JSON.parse(item.value.value.toString('utf8'))); }
            catch (_) {
results.push(item.value.value.toString('utf8'))
; }
        }
        if (item.done) { await iterator.close();
return results; }
    }
}

module.exports = LabResultsContract;
module.exports.contracts =
[LabResultsContract];
```

5.2 API Gateway – client/node-gateway/gateway.js

Author: Harsh Soni (Machine 3) | Language: Node.js
(@hyperledger/fabric-gateway, @grpc/grpc-js)

```
'use strict';

const crypto = require('crypto');
const fs = require('fs');
const path = require('path');
const grpc = require('@grpc/grpc-js');
const { connect, signers } =
require('@hyperledger/fabric-gateway');

const ROOT_DIR = path.resolve(__dirname, '..',
 '..');
const CCP_PATH = path.join(ROOT_DIR,
 'connection-profiles', 'lab-connection.json');
const USER_MSP_DIR = path.join(ROOT_DIR,
 'organizations', 'peerOrganizations',
 'lab.example.com', 'users',
 'User1@lab.example.com', 'msp');

// Static IP map to bypass DNS resolution
issues
const STATIC_IPS = {
  'peer0.lab.example.com': '10.166.46.138',
  'orderer.example.com': '10.166.46.138',
  'peer1.lab.example.com': '10.166.46.204',
  'peer2.lab.example.com': '127.0.0.1'
};

function newGrpcConnection(peerName,
connectionProfile) {
```

```

    const peer =
connectionProfile.peers[peerName];
    if (!peer) throw new Error(`Peer ${peerName}
not found`);
    const endpoint = new URL(peer.url);
    const tlsRootCert =
Buffer.from(peer.tlsCACerts.pem);
    const credentials =
grpc.credentials.createSsl(tlsRootCert);
    const ip = STATIC_IPS[endpoint.hostname] ||
endpoint.hostname;
    return new
grpc.Client(`${ip}:${endpoint.port}`,
credentials, {
      'grpc.ssl_target_name_override':
endpoint.hostname,
      'grpc.default_authority':
endpoint.hostname,
    });
}

function newIdentity() {
  const certPath = path.join(USER_MSP_DIR,
'signcerts', 'User1@lab.example.com-cert.pem');
  return { mspId: 'LabMSP', credentials:
fs.readFileSync(certPath) };
}

function newSigner() {
  const keyDir = path.join(USER_MSP_DIR,
'keystore');
  const privateKeyPem =
fs.readFileSync(path.join(keyDir,

```

```

fs.readdirSync(keyDir)[0]));
    return
    signers.newPrivateKeySigner(crypto.createPrivateKey(privateKeyPem));
}

async function withContract(peerName, callback)
{
    const profile =
JSON.parse(fs.readFileSync(CCP_PATH, 'utf8'));
    const client = newGrpcConnection(peerName ||
'peer0.lab.example.com', profile);
    const gateway = connect({
        client, identity: newIdentity(), signer:
newSigner(),
        evaluateOptions: () => ({ deadline:
Date.now() + 5000 }),
        endorseOptions: () => ({ deadline:
Date.now() + 15000 }),
        submitOptions: () => ({ deadline:
Date.now() + 15000 }),
        commitStatusOptions: () => ({ deadline:
Date.now() + 60000 }),
    });
    try {
        const contract =
gateway.getNetwork('ehrchannel').getContract('l
abresults');
        return await callback(contract, peerName);
    } finally { gateway.close(); client.close();
}
}

```

```
module.exports = { createLabResult,  
  getAllLabResults, getLabResultsByPatient,  
    listGatewayPeers, readLabResult,  
  updateLabStatus };
```

5.3 IPFS Integration – client/node-gateway/ipfs.js

Author: Prasoon Kumar(Machine 1) | Language: Node.js (native fetch API)

```
'use strict';
const crypto = require('crypto');

function getIpfsConfig() {
  return {
    apiUrl:      (process.env.IPFS_API_URL
  || 'http://127.0.0.1:5001').replace(/\$/ , ''),
    gatewayUrl: (process.env.IPFS_GATEWAY_URL
  || 'http://127.0.0.1:8080').replace(/\$/ , ''),
  };
}

async function addJsonToIpfs(payload, options =
{}) {
  const fileName = options.fileName ||
'payload.json';
  const jsonText = typeof payload === 'string'
? payload : JSON.stringify(payload);
  const formData = new FormData();
  formData.append('file', new Blob([jsonText],
{ type: 'application/json' }), fileName);
  const { apiUrl, gatewayUrl } =
getIpfsConfig();
  const pin = options.pin !== false;
  const resp = await
fetch(`${apiUrl}/api/v0/add?pin=${pin}&cid-vers
ion=1&raw-leaves=true`,
    { method: 'POST', body: formData });
  const addInfo = JSON.parse(await
```

```
resp.text()).trim().split('\n').pop());
  const digest =
crypto.createHash('sha256').update(jsonText).di
gest('hex');
  return { cid: addInfo.Hash, name:
addInfo.Name, size: Number(addInfo.Size),
  digest, gatewayUrl:
`${gatewayUrl}/ipfs/${addInfo.Hash}` };
}

async function readJsonFromIpfs(cid) {
  const { apiUrl, gatewayUrl } =
getIpfsConfig();
  const resp = await
fetch(`${apiUrl}/api/v0/cat?arg=${encodeURIComponent(
cid)}`,
    { method: 'POST' });
  const text = await resp.text();
  try { return { cid, data: JSON.parse(text),
gatewayUrl: `${gatewayUrl}/ipfs/${cid}` }; }
  catch(_) { return { cid, data: text,
gatewayUrl: `${gatewayUrl}/ipfs/${cid}` }; }
}

module.exports = { addJsonToIpfs,
getIpfsConfig, ipfsHealth, readJsonFromIpfs };
```

5.4 Express REST Server – client/node-gateway/server.js

Author: Harsh Soni (Machine 3) | Language: Node.js (Express)

```
'use strict';
const express = require('express');
const { createLabResult, getAllLabResults,
getLabResultsByPatient,
      listGatewayPeers, readLabResult,
updateLabStatus } = require('./gateway');
const { addJsonToIpfs, getIpfsConfig,
ipfsHealth, readJsonFromIpfs } =
require('./ipfs');
const { getAiAgentConfig,
runAgenticClinicalAnalysis } =
require('./ai-agent');

const app = express();
app.use(express.json({ limit: '1mb' }));

// — Key Endpoints

app.get('/api/meta',      (_,res)=>res.json({
peers: listGatewayPeers(),
  channel:'ehrchannel', chaincode:'labresults',
ipfs: getIpfsConfig() }));
app.get('/api/health',    (_,res)=>res.json({
ok: true }));
app.get('/api/ipfs/health', async(_,res)=>{
const h=await ipfsHealth();
  res.status(h.ok?200:503).json(h); });

// Records (pure on-chain)
app.get('/api/records',
```

```

async(req,res,next)=>{
  try { res.json(await
getAllLabResults({peerName:selectedPeer(req)}))
; } catch(e){next(e);} });
app.get('/api/records/:id',
async(req,res,next)=>{
  try { res.json(await
readLabResult(req.params.id,{peerName:selectedP
eer(req)})); } catch(e){next(e);} });
app.post('/api/records',
async(req,res,next)=>{
  try { res.status(201).json(await
createLabResult(req.body,{peerName:selectedPeer
(req)})); } catch(e){next(e);} });
app.patch('/api/records/:id/status',
async(req,res,next)=>{
  try { res.json(await
updateLabStatus(req.params.id,req.body.status,{
peerName:selectedPeer(req)})); }
catch(e){next(e);} });

// Hybrid: IPFS + blockchain
app.post('/api/records/ipfs',
async(req,res,next)=>{ /* stores payload on
IPFS, CID on-chain */ });

app.listen(Number(process.env.PORT||3000),
()=>console.log('Lab dashboard running at
http://localhost:3000'));

```

5.5 Docker Compose – Machine 1

(docker/docker-compose.machine1.yaml)

Configured by: Prasoon (Machine 1)

```
services:
  orderer.example.com:
    container_name: orderer.example.com
    image:
hyperledger/fabric-orderer:${FABRIC_VERSION}
    environment:
      -
ORDERER_GENERAL_LOCALMSPID=${ORDERER_MSP_ID}
      -
ORDERER_GENERAL_LISTENPORT=${ORDERER_PORT}
      - ORDERER_GENERAL_GENESISMETHOD=file
      - ORDERER_GENERAL_TLS_ENABLED=true
    volumes:
      -
../system-genesis-block/genesis.block:/var/hyperledger/orderer/orderer.genesis.block
    ports: ["7050:7050"]
    networks: [fabric]

  couchdb-peer0:
    image: couchdb:3.3
    environment: [COUCHDB_USER=admin,
COUCHDB_PASSWORD=adminpw]
    networks: [fabric]

  peer0.lab.example.com:
    container_name: peer0.lab.example.com
    image:
hyperledger/fabric-peer:${FABRIC_VERSION}
```

```

    environment:
      - CORE_PEER_ID=${PEER0_HOST}
      - CORE_PEER_ADDRESS=${PEER0_HOST}:7051
      - CORE_LEDGER_STATE_STATEDATABASE=CouchDB
      -
CORE_LEDGER_STATE_COUCHDBCONFIG_COUCHDBADDRESS=
couchdb-peer0:5984
      -
CORE_PEER_GOSSIP_EXTERNALENDPOINT=${PEER0_HOST}
:7051
      - CORE_PEER_LOCALMSPID=${LAB_MSP_ID}
      - CORE_PEER_TLS_ENABLED=true
    ports: ["7051:7051"]
    depends_on: [couchdb-peer0]
    networks: [fabric]

networks:
  fabric:
    name: ${FABRIC_NETWORK}

```

5.6 Docker Compose – Machine 2

([docker/docker-compose.machine2.yaml](#))

Configured by: Parag Jain (Machine 2)

```

services:
  couchdb-peer1:
    image: couchdb:3.3
    environment: [COUCHDB_USER=admin,
CORE_LEDGER_STATE_COUCHDBCONFIG_COUCHDBADDRESS=
couchdb-peer0:5984
CORE_PEER_GOSSIP_EXTERNALENDPOINT=${PEER0_HOST}
:7051
CORE_PEER_LOCALMSPID=${LAB_MSP_ID}
CORE_PEER_TLS_ENABLED=true
COUCHDB_PASSWORD=adminpw]
    networks: [fabric]

  peer1.lab.example.com:
    container_name: peer1.lab.example.com

```

```

    image:
hyperledger/fabric-peer:${FABRIC_VERSION}
    environment:
      - CORE_PEER_ID=${PEER1_HOST}
      - CORE_PEER_ADDRESS=${PEER1_HOST}:7051
      - CORE_LEDGER_STATE_STATEDATABASE=CouchDB
      -
CORE_LEDGER_STATE_COUCHDBCONFIG_COUCHDBADDRESS=
couchdb-peer1:5984
      -
CORE_PEER_GOSSIP_BOOTSTRAP=${LOCAL_BOOTSTRAP_PE
ER}
      -
CORE_PEER_GOSSIP_EXTERNALENDPOINT=${PEER1_HOST}
:7051
      - CORE_PEER_LOCALMSPID=${LAB_MSP_ID}
      - CORE_PEER_TLS_ENABLED=true
    ports: ["7051:7051"]
    depends_on: [couchdb-peer1]
    networks: [fabric]

networks:
  fabric:
    name: ${FABRIC_NETWORK}

```

5.7 Docker Compose – Machine 3

([docker/docker-compose.machine3.yaml](#))

Configured by: Harsh Soni (Machine 3)

```

services:
  couchdb-peer2:
    image: couchdb:3.3
    environment: [COUCHDB_USER=admin,

```

```

COUCHDB_PASSWORD=adminpw]
  networks: [fabric]

  peer2.lab.example.com:
    container_name: peer2.lab.example.com
    image:
hyperledger/fabric-peer:${FABRIC_VERSION}
    environment:
      - CORE_PEER_ID=${PEER2_HOST}
      - CORE_PEER_ADDRESS=${PEER2_HOST}:7051
      - CORE_LEDGER_STATE_STATEDATABASE=CouchDB
      -
CORE_LEDGER_STATE_COUCHDBCONFIG_COUCHDBADDRESS=
couchdb-peer2:5984
      -
CORE_PEER_GOSSIP_BOOTSTRAP=${LOCAL_BOOTSTRAP_PE
ER}
      -
CORE_PEER_GOSSIP_EXTERNALENDPOINT=${PEER2_HOST}
:7051
      - CORE_PEER_LOCALMSPID=${LAB_MSP_ID}
      - CORE_PEER_TLS_ENABLED=true
    ports: ["7051:7051"]
    depends_on: [couchdb-peer2]
    extra_hosts:
      - "orderer.example.com:10.166.46.138"
      - "peer0.lab.example.com:10.166.46.138"
      - "peer1.lab.example.com:10.166.46.204"
    networks: [fabric]

networks:
  fabric:

```

```
name: ${FABRIC_NETWORK}
```

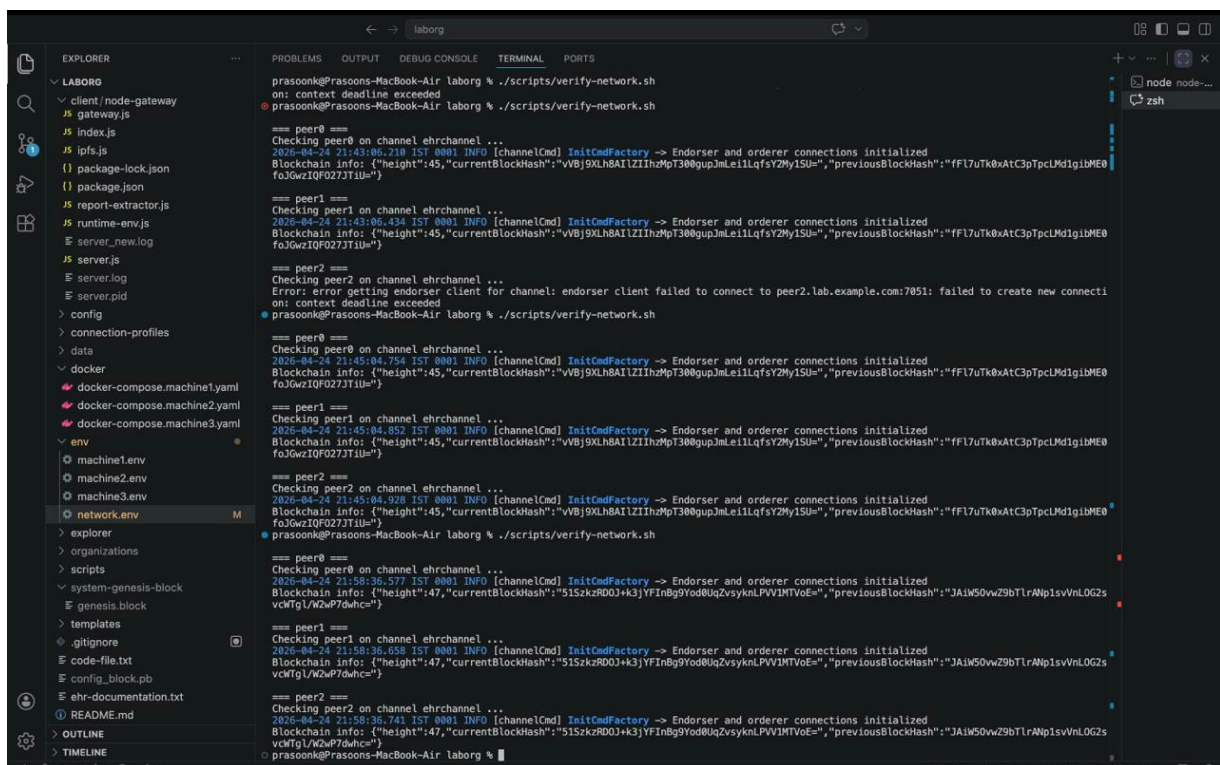
6. Verification and Results

6.1 Ledger Synchronisation Verification

After all three peers joined ehrchannel and installed the labresults chaincode, the following verification was performed from Machine 1:

```
./scripts/verify-network.sh
./scripts/query-labresults.sh peer0
GetAllLabResults
./scripts/query-labresults.sh peer1
GetAllLabResults
./scripts/query-labresults.sh peer2
GetAllLabResults
```

Paste ledger synchronisation output from all three peers here:



```
praseonk@Praseons-MacBook-Air laborg % ./scripts/verify-network.sh
on: context deadline exceeded
praseonk@Praseons-MacBook-Air laborg % ./scripts/verify-network.sh

=== peer0 ===
Checking peer0 on channel ehrchannel ...
2026-04-24 21:43:06.210 IST 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
Blockchain info: {"height":45,"currentBlockHash":"vBj9XLh8A1Z1IhzMPT300gUpJmLeilQfsY2My1SU=", "previousBlockHash":"fF1uTk0xAtC3pTpcLmd1g1bME0foJGwzIQf027JT1U="}

=== peer1 ===
Checking peer1 on channel ehrchannel ...
2026-04-24 21:43:06.434 IST 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
Blockchain info: {"height":45,"currentBlockHash":"vBj9XLh8A1Z1IhzMPT300gUpJmLeilQfsY2My1SU=", "previousBlockHash":"fF1uTk0xAtC3pTpcLmd1g1bME0foJGwzIQf027JT1U="}

=== peer2 ===
Checking peer2 on channel ehrchannel ...
Error: error getting endorser client for channel: endorser client failed to connect to peer2.lab.example.com:7051: failed to create new connecti
on: context deadline exceeded
praseonk@Praseons-MacBook-Air laborg % ./scripts/verify-network.sh

=== peer0 ===
Checking peer0 on channel ehrchannel ...
2026-04-24 21:45:04.754 IST 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
Blockchain info: {"height":45,"currentBlockHash":"vBj9XLh8A1Z1IhzMPT300gUpJmLeilQfsY2My1SU=", "previousBlockHash":"fF1uTk0xAtC3pTpcLmd1g1bME0foJGwzIQf027JT1U="}

=== peer1 ===
Checking peer1 on channel ehrchannel ...
2026-04-24 21:45:04.852 IST 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
Blockchain info: {"height":45,"currentBlockHash":"vBj9XLh8A1Z1IhzMPT300gUpJmLeilQfsY2My1SU=", "previousBlockHash":"fF1uTk0xAtC3pTpcLmd1g1bME0foJGwzIQf027JT1U="}

=== peer2 ===
Checking peer2 on channel ehrchannel ...
2026-04-24 21:45:04.928 IST 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
Blockchain info: {"height":45,"currentBlockHash":"vBj9XLh8A1Z1IhzMPT300gUpJmLeilQfsY2My1SU=", "previousBlockHash":"fF1uTk0xAtC3pTpcLmd1g1bME0foJGwzIQf027JT1U="}

praseonk@Praseons-MacBook-Air laborg % ./scripts/verify-network.sh

=== peer0 ===
Checking peer0 on channel ehrchannel ...
2026-04-24 21:58:36.577 IST 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
Blockchain info: {"height":47,"currentBlockHash":"51SzkzRD0J+k3jYfInBg9Yod8Uq2vzyknlPVV1MTVoE=", "previousBlockHash":"JAiW50vwZ9bTlRAnp1svVnLOG2sVcMtgLW2wP7dWdc="}

=== peer1 ===
Checking peer1 on channel ehrchannel ...
2026-04-24 21:58:36.658 IST 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
Blockchain info: {"height":47,"currentBlockHash":"51SzkzRD0J+k3jYfInBg9Yod8Uq2vzyknlPVV1MTVoE=", "previousBlockHash":"JAiW50vwZ9bTlRAnp1svVnLOG2sVcMtgLW2wP7dWdc="}

=== peer2 ===
Checking peer2 on channel ehrchannel ...
2026-04-24 21:58:36.741 IST 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
Blockchain info: {"height":47,"currentBlockHash":"51SzkzRD0J+k3jYfInBg9Yod8Uq2vzyknlPVV1MTVoE=", "previousBlockHash":"JAiW50vwZ9bTlRAnp1svVnLOG2sVcMtgLW2wP7dWdc="}

praseonk@Praseons-MacBook-Air laborg %
```

```
Problems Output Debug Console Terminal Ports
harsh@harsh-GF63-Thin-95CXR:~/Downloads/laborg$ cd laborg
harsh@harsh-GF63-Thin-95CXR:~/Downloads/laborg/laborg$ ./scripts/start-machine3.sh
[+] up 3/3
  ✓ Network fabric labnet Created 0.1s
  ✓ Container couchdb-peer2.lab.example.com Created 0.1s
  ✓ Container peer2.lab.example.com Created 0.1s
NAMEs STATUS PORTS
peer2.lab.example.com Up Less than a second 0.0.0.0:7051->7051/tcp, [::]:7051->7051/tcp
couchdb-peer2.lab.example.com Up 1 second 4369/tcp, 5984/tcp, 9100/tcp
harsh@harsh-GF63-Thin-95CXR:~/Downloads/laborg/laborg$ ./scripts/verify-network.sh

=== peer0 ===
Checking peer0 on channel ehrchannel ...
2026-04-24 21:44:54.130 IST 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
Blockchain info: {"height":45,"currentBlockHash":"vVBj9XLH8A1lZiIhZmpT300gUpJmLe1lQfsY2My15U=", "previousBlockHash":"fFL7uTk8xATC3pTpcLMD1g1bME0foJ0wzIQF027JT1U="}

=== peer1 ===
Checking peer1 on channel ehrchannel ...
2026-04-24 21:44:54.203 IST 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
Blockchain info: {"height":45,"currentBlockHash":"vVBj9XLH8A1lZiIhZmpT300gUpJmLe1lQfsY2My15U=", "previousBlockHash":"fFL7uTk8xATC3pTpcLMD1g1bME0foJ0wzIQF027JT1U="}

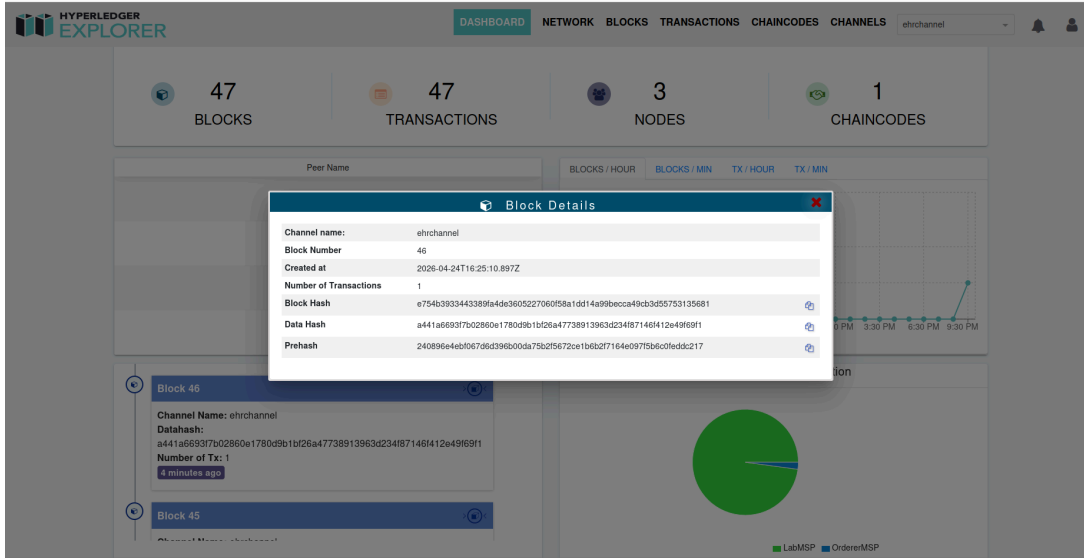
=== peer2 ===
Checking peer2 on channel ehrchannel ...
2026-04-24 21:44:54.389 IST 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
Blockchain info: {"height":45,"currentBlockHash":"vVBj9XLH8A1lZiIhZmpT300gUpJmLe1lQfsY2My15U=", "previousBlockHash":"fFL7uTk8xATC3pTpcLMD1g1bME0foJ0wzIQF027JT1U="}
harsh@harsh-GF63-Thin-95CXR:~/Downloads/laborg/laborg$ docker ps
CONTAINER ID IMAGE PORTS NAMES
d939ec244eb dev-peer2.lab.example.com-labresults-2-d9b294cbde110f932e053468bd2cb64a171c84b51115078a957c93b702685362-707ae9efb83e02ef6a9c81e591e82a269407f055eb6e6e8c9abc5d5a6992f 18 seconds
0605c64ba26 hyperledger/fabric-peer:2.5.15 dev-peer2.lab.example.com-labresults-2-d9b294cbde110f932e053468bd2cb64a171c84b51115078a957c93b702685362 "peer node start" 27 seconds
7c8214365f8a couchdb:3.3 peer2.lab.example.com "tini -- /docker-ent-" 27 seconds
ago Up 26 seconds 0.0.0.0:7051->7051/tcp, [::]:7051->7051/tcp couchdb-peer2.lab.example.com
```

6.2 CouchDB State Verification

```
# Access CouchDB on Machine 1 (peer0):
curl -u admin:adminpw
http://localhost:5984/ehrchannel_labresults/_all_docs?include_docs=true

# Check block height:
peer channel getinfo -c ehrchannel
```

Paste CouchDB query output and block height here:



6.3 API Gateway Verification

```
curl http://localhost:3000/api/health
curl http://localhost:3000/api/meta
curl http://localhost:3000/api/records
curl -X POST http://localhost:3000/api/records \
  -H "Content-Type: application/json" \
  -d '{
    "resultId": "labresult5",
    "patientId": "patient-1005",
    "testCode": "LIPID",
    "collectedAt": "2026-04-24T10:00:00Z",
    "status": "REPORTED",
    "resultData": {
      "hdl": "55",
      "ldl": "105",
      "triglycerides": "140"
    }
  }'
```

Paste API gateway test output here:

Create Record

Result ID

e.g. labresult10

Patient ID

e.g. patient-1010

Test Code

e.g. LIPD

Collected At

mm / dd / yyyy. - - -

Initial Status

e.g. PENDING or REPORTED

Result Data (JSON, optional with file upload)

{ "hdi": "56", "id1": "168" }

Lab Report File (PDF/Image)

Browse... No file selected.

Upload a PDF/image report to store in IPFS and extract text for AI analysis.

☒ Store result payload in IPFS (CID anchored on-chain)

☒ Run Gemini AI agents (lab analysis + decision support) before submit

Submit Record

Ledger Snapshot

peer2.lab.example.com - 25 record(s)

Result ID	Patient ID	Test Code	Status	Collected At
labresult51	patient-1117	CBC	PENDING	2026-04-24T16:23:00.000Z

Transaction Output

Clear Logs

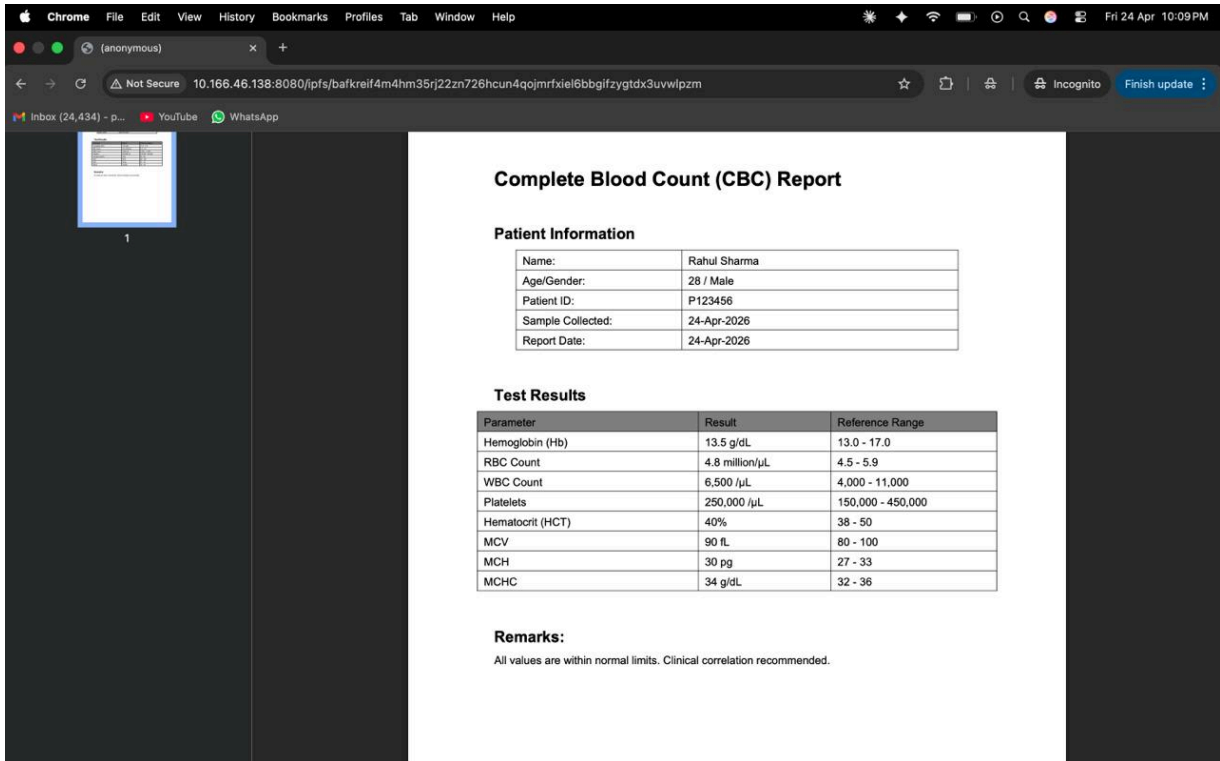
GET /api/records/labresult51

```
{
  "peer": "peer2.lab.example.com",
  "result": {
    "collectedAt": "2026-04-24T16:23:00.000Z",
    "docType": "labResult",
    "ownerOrg": "LabMSP",
    "patientId": "patient-1117",
    "resultData": {
      "ai": {
        "confidence": {
          "decisionSupport": "low",
          "labReport": "low"
        },
        "disclaimer": "AI output is assistive only and must be reviewed by a licensed clinician.",
        "enabled": true,
        "generatedAt": "2026-04-24T16:25:10.292Z",
        "model": "gemini-2.5-flash",
        "ok": false,
        "provider": "gemini",
        "recommendation": "Review historical trends and correlate with current symptoms before treatment decisions.",
        "summary": "Basic heuristic analysis completed. No strong abnormality inferred from keyword scan.",
        "summaryId": "ba1fe1b1c7e0a8c7f1e5a8b7f7d6d9a1jwcazmit2qpeuclbny",
        "summaryDigest": "212888d049592a44e09417b6ffa353165c401b731c0fab3c4c451018e",
        "warning": "Gemini call failed: Gemini API request failed for model gemini-2.5-flash-lite via https://..."
      }
    },
    "byteLength": 441,
    "cid": "bafkreifamhnc2n47kqmnaqnbtpqbygw7qn3mehkyjvsyuagkhpae",
    "digest": "a0804ec16bcbfaa8cb41b086ef0f7d0b11ef376c7f16ear2655c4a94328ef21",
    "gatewayUrl": "http://10.106.4b.138:8080/ips/bafkreifamhnc2n47kqmnaqnbtpqbygw7qn3mehkyjvsyuagk",
    "reportFile": {
      "byteLength": 2503,
      "digest": "a0804ec16bcbfaa8cb41b086ef0f7d0b11ef376c7f16ear2655c4a94328ef21"
    }
  }
}
```

6.4 IPFS Integration Verification

```
curl http://localhost:3000/api/ipfs/health
curl -X POST
http://localhost:3000/api/records/ipfs \
-H "Content-Type: application/json" \
-d
'{"resultId":"labresult6","patientId":"patient-
1006","testCode":"CBC",
"collectedAt":"2026-04-24T11:00:00Z","status":"
REPORTED",
"resultData":{"hemoglobin":"14.2","wbc":"7000",
"platelets":"280000"}}'
```

Paste IPFS health and hybrid record creation output here:



The screenshot shows a web browser window with a dark theme. The address bar displays a Not Secure connection to a local IPFS node. The page content is a 'Complete Blood Count (CBC) Report' for a patient named Rahul Sharma. The report includes a table for Patient Information and a table for Test Results. The Test Results table shows values for Hemoglobin, RBC Count, WBC Count, Platelets, Hematocrit, MCV, MCH, and MCHC, all of which are within the reference range.

Complete Blood Count (CBC) Report

Patient Information

Name:	Rahul Sharma
Age/Gender:	28 / Male
Patient ID:	P123456
Sample Collected:	24-Apr-2026
Report Date:	24-Apr-2026

Test Results

Parameter	Result	Reference Range
Hemoglobin (Hb)	13.5 g/dL	13.0 - 17.0
RBC Count	4.8 million/ μ L	4.5 - 5.9
WBC Count	6,500 / μ L	4,000 - 11,000
Platelets	250,000 / μ L	150,000 - 450,000
Hematocrit (HCT)	40%	38 - 50
MCV	90 fL	80 - 100
MCH	30 pg	27 - 33
MCHC	34 g/dL	32 - 36

Remarks:
All values are within normal limits. Clinical correlation recommended.

7. Conclusion and Future Scope

7.1 Conclusion

This project successfully demonstrates a production-grade pattern for managing Electronic Health Records using Hyperledger Fabric distributed across three physical machines. The three-peer topology provides:

- Decentralisation: No single point of failure; any peer can serve read requests.
- Immutability: All write transactions are ordered and committed to all peers.
- Confidentiality: TLS mutual authentication and MSP-based identity enforcement.

- Scalability: Off-chain IPFS storage prevents blockchain bloat for large data.
- Interoperability: REST API makes the blockchain network accessible from any HTTP client.

7.2 Individual Achievement Summary

Team Member	Machine / Role	Key Deliverables
Prasoon Kumar (25CSM2R15)	Machine 1 Infrastructure Architect	Orderer setup, genesis block, channel creation, crypto distribution, chaincode lifecycle, deployment automation
Parag Jain (25CSM2R27)	Machine 2 Smart Contract Lead	Chaincode (index.js), CouchDB rich queries, endorsement policy, patient portal, clinical portals
Harsh Soni (25CSM2S16)	Machine 3 API Gateway Architect	gateway.js, ipfs.js, server.js, REST API, static IP routing, IPFS hybrid storage, web dashboard

7.3 Future Scope

- Migrate from cryptogen to Fabric CA for dynamic identity enrollment and revocation.
- Add multi-organisation support (Hospital MSP, Pharmacy MSP, Insurance MSP) with separate channels.
- Implement private data collections (PDC) for confidential inter-org data sharing.
- Integrate a dedicated IPFS cluster (instead of a local daemon) for production off-chain storage.

- Add role-based access control (RBAC) in chaincode using client identity attributes.
- Deploy on Kubernetes using Hyperledger Fabric Operator for horizontal scaling.
- Implement real-time ledger event streaming via Fabric event hub to notify downstream systems.
- Conduct formal security audit: key management, TLS certificate rotation, and peer firewall rules.